CLASIFICADOR DE IMAGENES

BRIGITH / Fondo con VGG-16

Informe del entrenamiento, resultados, aplicacion de prueba y codigo fuente



Foto de prueba compartida por la usuaria

Fecha del informe: 9 de noviembre de 2026

1. Resumen del proyecto

El sistema clasifica una imagen en una de dos categorias: **Fondo** o **BRIGITH**. Se entreno una cabeza clasificadora sobre VGG-16 preentrenada con ImageNet y se preparo una interfaz web local para probar fotografias.

2. Datos y entrenamiento

Imagenes leidas: 89 de Fondo y 77 de BRIGITH (166 en total). Durante la carga se informo que 10 archivos no pudieron leerse. Las imagenes se redimensionaron a 224 x 224; se uso aumento de datos y una division estratificada de entrenamiento y prueba.

Modelo: VGG-16 con pesos ImageNet y base convolucional congelada; capas densas para las dos clases. Maximo de 50 epocas, lotes de 16 y parada temprana si no mejora la perdida de validacion.

3. Evaluacion registrada

Accuracy informada: 100 % en el conjunto de prueba de 34 imagenes. **Perdida:** 0.0032. **Matriz de confusion:** [[18, 0], [0, 16]].

Este resultado describe un conjunto de prueba pequeno y no garantiza el mismo rendimiento con fotografias nuevas. Una confianza de 100 % es la salida del modelo, no una certeza absoluta.

4. Prueba de la fotografia nueva

La aplicacion clasifico IMG_2855.PNG como **BRIGITH**, con confianza reportada de **100 %** (Fondo: 0 %; BRIGITH: 100 %). La prediccion solo indica a cual de las dos clases aprendidas se parece la imagen.

5. Como abrir y probar la aplicacion

Con Streamlit instalado en el entorno Python utilizado para el proyecto, abre PowerShell y ejecuta:

```
& "C:\p\v\Scripts\streamlit.exe" run "C:\Users\Usuario\Documents\2026\BRIGITH\MAESTRIA\
FUNDAMENTOS DE INTELIGENCIA ARTIFICIAL\SEMANA 3\ACTIVIDAD 8\app.py"
```

Luego abre **http://localhost:8501**, selecciona una imagen JPG, JPEG, PNG, BMP o WEBP y consulta la prediccion y las probabilidades. La direccion es local; solo funciona mientras Streamlit siga ejecutandose en esa computadora.

Archivos del modelo: **modelo_brigith_fondo.keras** y **clases.json**. Grafica del entrenamiento: **curvas_entrenamiento.png**.

Anexo A. Codigo de entrenamiento

Fuente: C:\Users\Usuario\Downloads\entrenar_modelo.py

```
"""
Actividad 8 - Clasificador "BRIGITH" vs "Fondo" con VGG-16 (transfer learning)
Ejecutar:  python entrenar_modelo.py
Genera:    modelo_brigith_fondo.keras  y  clases.json
"""
import os, json
import cv2
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.applications import VGG16
from tensorflow.keras.applications.vgg16 import preprocess_input
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.utils.class_weight import compute_class_weight

# ------------------------------------------------------------------------
# 1. CONFIGURACIÓN (aquí cambias los hiperparámetros para la parte opcional)
# ------------------------------------------------------------------------
BASE = r'C:/Users/Usuario/Documents/2026/BRIGITH/MAESTRIA/FUNDAMENTOS DE INTELIGENCIA
ARTIFICIAL/SEMANA 3/ACTIVIDAD 8'
CARPETA_ESTUDIANTE = os.path.join(BASE, 'BRIGITH')   # Label 1
CARPETA_FONDOS     = os.path.join(BASE, 'FONDOS')    # Label 0
NOMBRE_ESTUDIANTE  = 'BRIGITH'

IMG_SIZE   = 224
EPOCHS     = 50           # hiperparámetro: épocas (EarlyStopping corta antes si no mejora)
BATCH_SIZE = 16           # hiperparámetro: tamaño de lote
LOSS       = 'categorical_crossentropy'   # hiperparámetro: función de pérdida
OPTIMIZER  = tf.keras.optimizers.Adam(learning_rate=1e-4)  # hiperparámetro: optimizador
AUGMENTATION = True       # hiperparámetro: aumento de imágenes
SEED = 42

# Orden de clases: índice 0 = Fondo, índice 1 = estudiante
CLASES = ['Fondo', NOMBRE_ESTUDIANTE]

# ------------------------------------------------------------------------
# 2. CARGA DE IMÁGENES
# ------------------------------------------------------------------------
EXT = ('.jpg', '.jpeg', '.png', '.bmp', '.webp')

def cargar_carpeta(carpeta, etiqueta):
    imgs, labs = [], []
    for nombre in sorted(os.listdir(carpeta)):
        if not nombre.lower().endswith(EXT):
            continue
        # np.fromfile + imdecode evita problemas de OpenCV con rutas con tildes/espacios en
Windows
        datos = np.fromfile(os.path.join(carpeta, nombre), dtype=np.uint8)
        img = cv2.imdecode(datos, cv2.IMREAD_COLOR)
        if img is None:
            print('No se pudo leer:', nombre)
            continue
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)          # RGB (igual que en la app)
        img = cv2.resize(img, (IMG_SIZE, IMG_SIZE))
        imgs.append(img)
        labs.append(etiqueta)
    return imgs, labs

x_f, y_f = cargar_carpeta(CARPETA_FONDOS, 0)
x_e, y_e = cargar_carpeta(CARPETA_ESTUDIANTE, 1)
print(f'Fondos: {len(x_f)} | {NOMBRE_ESTUDIANTE}: {len(x_e)}')

images = np.array(x_f + x_e, dtype=np.float32)
labels = np.array(y_f + y_e)
labels_oh = tf.keras.utils.to_categorical(labels, num_classes=2)

# ------------------------------------------------------------------------
# 3. DIVISIÓN TRAIN / TEST (estratificada para mantener la proporción de clases)
# ------------------------------------------------------------------------
X_train, X_test, y_train, y_test = train_test_split(
    images, labels_oh, test_size=0.2, stratify=labels, random_state=SEED)

# Preprocesamiento propio de VGG-16 (¡debe ser idéntico en la app!)
X_train = preprocess_input(X_train.copy())
X_test  = preprocess_input(X_test.copy())

# Pesos de clase por si hay más fondos que fotos tuyas (o al revés)
pesos = compute_class_weight('balanced', classes=np.array([0, 1]), y=np.argmax(y_train,
axis=1))
class_weight = {0: pesos[0], 1: pesos[1]}
print('class_weight:', class_weight)

# ------------------------------------------------------------------------
# 4. MODELO: VGG-16 congelada + cabeza densa (+ aumento de imágenes en el modelo)
# ------------------------------------------------------------------------
```

```
aug = models.Sequential([
    layers.RandomFlip('horizontal'),
    layers.RandomRotation(0.08),
    layers.RandomZoom(0.15),
    layers.RandomTranslation(0.1, 0.1),
    layers.RandomContrast(0.2),
    layers.RandomBrightness(0.2, value_range=(-128, 128)),  # rango de preprocess_input
], name='aumento')  # solo actúan durante el entrenamiento

base = VGG16(weights='imagenet', include_top=False, input_shape=(IMG_SIZE, IMG_SIZE, 3))
base.trainable = False

entrada = layers.Input((IMG_SIZE, IMG_SIZE, 3))
x = aug(entrada) if AUGMENTATION else entrada
x = base(x, training=False)
x = layers.Flatten()(x)
x = layers.Dense(256, activation='relu')(x)
x = layers.Dropout(0.5)(x)
salida = layers.Dense(2, activation='softmax')(x)
model = models.Model(entrada, salida)

model.compile(loss=LOSS, optimizer=OPTIMIZER, metrics=['accuracy'])
model.summary()

early = EarlyStopping(monitor='val_loss', patience=8, restore_best_weights=True)
hist = model.fit(X_train, y_train, validation_data=(X_test, y_test),
                 epochs=EPOCHS, batch_size=BATCH_SIZE,
                 class_weight=class_weight, callbacks=[early])

# ------------------------------------------------------------------------
# 5. EVALUACIÓN
# ------------------------------------------------------------------------
loss, acc = model.evaluate(X_test, y_test, verbose=0)
print(f'\nLoss: {loss:.4f} | Accuracy: {acc:.4f}')
pred = np.argmax(model.predict(X_test), axis=1)
real = np.argmax(y_test, axis=1)
print(classification_report(real, pred, target_names=CLASES))
print('Matriz de confusión:\n', confusion_matrix(real, pred))

plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1); plt.plot(hist.history['loss'], label='train'); plt.plot(hist.
history['val_loss'], label='val'); plt.title('Loss'); plt.legend()
plt.subplot(1, 2, 2); plt.plot(hist.history['accuracy'], label='train'); plt.plot(hist.
history['val_accuracy'], label='val'); plt.title('Accuracy'); plt.legend()
plt.tight_layout(); plt.savefig('curvas_entrenamiento.png'); plt.show()

# ------------------------------------------------------------------------
# 6. GUARDAR MODELO + NOMBRES DE CLASES
# ------------------------------------------------------------------------
model.save('modelo_brigith_fondo.keras')
with open('clases.json', 'w', encoding='utf-8') as f:
    json.dump(CLASES, f)
print('Modelo guardado: modelo_brigith_fondo.keras')

# ------------------------------------------------------------------------
# 7. PRUEBA RÁPIDA CON UNA IMAGEN NUEVA (opcional): cambia la ruta
# ------------------------------------------------------------------------
def predecir(ruta):
    img = cv2.imdecode(np.fromfile(ruta, dtype=np.uint8), cv2.IMREAD_COLOR)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    x = preprocess_input(cv2.resize(img, (IMG_SIZE, IMG_SIZE)).astype(np.float32)[None])
    p = model.predict(x, verbose=0)[0]
    plt.imshow(img); plt.axis('off'); plt.title(f'{CLASES[int(np.argmax(p))]}  ({p.max():.
1%})'); plt.show()
    return p

# predecir(r'C:/ruta/a/foto_nueva.jpg')
```

Anexo B. Codigo de la aplicacion web

Fuente: C:\Users\Usuario\Documents\2026\BRIGITH\MAESTRIA\FUNDAMENTOS DE INTELIGENCIA ARTIFICIAL\SEMANA 3\ACTIVIDAD 8\app.py

```
import json
from pathlib import Path

import cv2
import numpy as np
import streamlit as st
import tensorflow as tf
from tensorflow.keras.applications.vgg16 import preprocess_input


BASE_DIR = Path(__file__).resolve().parent
MODEL_PATH = BASE_DIR / "modelo_brigith_fondo.keras"
CLASSES_PATH = BASE_DIR / "clases.json"
IMAGE_SIZE = 224


@st.cache_resource
def load_model():
    if not MODEL_PATH.is_file():
        raise FileNotFoundError(f"No se encontró el modelo: {MODEL_PATH}")
    return tf.keras.models.load_model(MODEL_PATH)


@st.cache_data
def load_classes():
    if not CLASSES_PATH.is_file():
        raise FileNotFoundError(f"No se encontró el archivo de clases: {CLASSES_PATH}")
    with CLASSES_PATH.open("r", encoding="utf-8") as file:
        classes = json.load(file)
    if not isinstance(classes, list) or not classes:
        raise ValueError("clases.json debe contener una lista de nombres de clase.")
    return classes


st.set_page_config(page_title="Clasificador BRIGITH / Fondo", page_icon="■■")
st.title("Clasificador de imágenes")
st.write("Carga una fotografía para saber si el modelo la clasifica como BRIGITH o Fondo.")

uploaded_file = st.file_uploader(
    "Selecciona una imagen",
    type=["jpg", "jpeg", "png", "bmp", "webp"],
)

if uploaded_file is not None:
    image_bytes = np.frombuffer(uploaded_file.getvalue(), dtype=np.uint8)
    image_bgr = cv2.imdecode(image_bytes, cv2.IMREAD_COLOR)

    if image_bgr is None:
        st.error("No se pudo leer el archivo. Prueba con otra imagen.")
    else:
        image_rgb = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2RGB)
        st.image(image_rgb, caption="Imagen seleccionada", use_container_width=True)

        try:
            model = load_model()
            classes = load_classes()
            resized = cv2.resize(image_rgb, (IMAGE_SIZE, IMAGE_SIZE))
            batch = np.expand_dims(resized.astype(np.float32), axis=0)
            prediction = model.predict(preprocess_input(batch), verbose=0)[0]

            if len(prediction) != len(classes):
                raise ValueError(
                    f"El modelo devuelve {len(prediction)} clases, "
                    f"pero clases.json contiene {len(classes)}."
                )

            best_index = int(np.argmax(prediction))
            st.subheader(f"Predicción: {classes[best_index]}")
            st.metric("Confianza", f"{prediction[best_index]:.1%}")

            st.write("Probabilidad por clase:")
            for class_name, probability in zip(classes, prediction):
                st.write(f"{class_name}: {probability:.1%}")
        except (FileNotFoundError, ValueError, OSError) as error:
            st.error(str(error))
```